

Future work and recommendations

Printed from `docs/FUTURE_WORK.md`. The complete handover, with every other document, is `docs/HANDOVER.pdf`.

Future work and recommendations

DOCS/FUTURE_WORK.MD

Future work and recommendations

Status (7 October 2026): the software phase is complete and the repository is public. Phases 0 to 6 of `docs/SPECIFICATION.md` passed (tags `phase-0` to `phase-6`), the Kaggle-trained obstacle-avoidance policy is in use, the first public release is tagged `v1.0.0`, and a GitHub check runs on every push. This note lists what can come next, what I recommend, and roughly how much work each item is. Effort figures are estimates, not measurements.

In short:

- **One-click setup is possible.** Level 1 is done: `./setup.sh` installs everything the app, the fast simulator and the tests need, in about 3 minutes, and a GitHub check runs it on every push (section 1). The full PX4 + ROS 2 system can be one command with a container (Docker), or one click in the browser with GitHub Codespaces, in about 2–3 days.
- **Public since 7 October 2026.** The licence (Apache-2.0), the secrets scan and the responsible-use note were done first (section 2).
- **Onboard code in C++:** the policy's inference is ported to C++17 and checked against the Python code. The next step is a ROS 2 node around it (section 5).
- **Swarm logic:** stop a drone with a patchy link from grabbing the lead back, and keep drones apart when they cannot hear each other.
- **RL:** the next run should change the training set-up, not just run longer.
- **Real drones:** hardware, a stable PX4 release, bench tests, then the flight test plan, only with legal permission.

1. One-click setup for anyone who copies the repository

Can it be done? Yes, in layers. Today a new person has to follow 7 manual steps in `docs/RUNBOOK.md`, section 1. They install ROS 2 Jazzy and MAVROS, fetch the GeographicLib data, clone and build PX4, install the Python packages, set up the RL environment, add map keys and build the one-click launcher. Some of that needs `sudo`, the PX4 build takes about 15 GB of disk, and one download (GeographicLib from SourceForge) failed on this laptop and had to be done by hand.

Not everyone needs all of it. Most visitors only want to see the swarm fly in the app, which needs none of the heavy parts. So the setup comes in three levels; **level 1 is built** (29 September 2026), levels 2 and 3 are planned:

#	What the person gets	How they start it	What they need	Their waiting time	Work to build it
1	Quick start (done): the ground-control app (2-D map, 3-D view, faults, results), the fast simulator with the real agent code, the RL policy, all tests	<code>./setup.sh</code> , then the app opens in the browser	Linux or macOS with Python 3.12 (Windows through WSL); no <code>sudo</code>	about 3 minutes (measured on a fresh copy)	done

#	What the person gets	How they start it	What they need	Their waiting time	Work to build it
2	Full install script: everything, including 10 PX4 drones with MAVROS	<code>bash scripts/install_all.sh</code>	Ubuntu 24.04, 15 GB free disk, 8 GB RAM, the <code>sudo</code> password once	about 1–1.5 hours (downloads and the PX4 build)	about 1–2 days, tested on a clean Ubuntu 24.04 virtual machine
3	Container: the same as level 2, identical on every machine	<code>docker compose up</code> , or the "Open in Codespaces" button on GitHub (nothing installed at all)	Docker, or a GitHub account (Codespaces has a monthly free allowance; 10 PX4 drones need a 4-core machine)	about 10–20 minutes to download a ready image	about 2–3 days

What each level would do: - **Level 1, `setup.sh` (built):** 1. Checks Python 3.12. 2. Creates `.venv` and installs `requirements.txt` (numpy, scipy, matplotlib, PyYAML, psutil, pytest; the tested versions are written in the file). 3. Runs the tests. 4. Creates an empty `config/map_keys.local.yaml` for the user's own keys. 5. Starts the app on port 8080 and opens the browser. PyTorch is not needed here: the policy runs on plain numpy. Tested on a fresh copy on 29 September 2026: about 3 minutes, all tests passed, every page of the app loaded. - **Level 2, `install_all.sh`:** 1. Check Ubuntu 24.04, free disk (at least 15 GB) and memory. 2. Install ROS 2 Jazzy, MAVROS and the GeographicLib data. Keep a second download source, because SourceForge failed once. 3. Clone PX4 at `v1.18.0-rc1` and build `px4_sitl_sih` with 2 jobs (`scripts/build_px4.sh`). 4. Do everything level 1 does, plus the optional RL training environment. 5. Build the one-click launcher (`scripts/install_launcher.sh`). 6. Finish with `scripts/env_audit.sh` and a one-drone take-off and landing, the Phase 0 test. The script must be safe to run twice and must say clearly which step failed. - **Level 3, container:** - A `Dockerfile` starting from the official ROS 2 Jazzy image, with MAVROS, the GeographicLib data and PX4 SIH built inside. - A `docker-compose.yaml` that starts the app on port 8080. - A `.devcontainer/devcontainer.json` for Codespaces and VS Code. - Build the image once and publish it on GitHub's container registry, so users download it instead of building it for an hour.

The automatic check (GitHub Actions) is on. `.github/workflows/quick-start.yaml` runs after every push to `main` and every pull request. On a fresh copy, without PX4 or ROS 2, it:

1. runs `./setup.sh` (install and tests);
2. starts the app and checks that it serves the swarm state;
3. checks that the C++ weight file matches the trained policy;
4. builds the C++ policy module and tests it against the Python reference.

The README shows the result as a badge. The check was written on 29 September 2026 but stayed switched off while the repository was private, because GitHub would not start it for this account; Actions is free for public repositories. When the container exists, add its build to the same check. Without such a check, setup scripts break quietly as packages change.

What cannot be made automatic: - **Map keys** (Mapbox, Cesium ion) belong to a person's own free account and must never be shipped. The 2-D map works without them (OpenStreetMap streets). Satellite imagery needs a Mapbox key, and the 3-D view needs a Cesium ion key. The setup can ask for them and write `config/map_keys.local.yaml`. - **The `sudo` password** for a native install (level 2). Containers avoid it. - **Kaggle training** runs on the user's own Kaggle account (`docs/KAGGLE_GUIDE.md`). - **Computer limits:** 10 PX4 drones used about 1.1 GB of RAM and 77 % of this laptop's CPU. Weaker machines should start with 3 drones.

Recommendation: level 1 is done. Build level 3 next, because a container is the only way to make the full PX4 system truly one step on any computer. Keep level 2 for people who want a native install, for example on a drone's companion computer.

2. The public repository: what was done, what is left

The repository is public since 7 October 2026 (release `v1.0.0`).

- **Licence: done** (29 September 2026). Apache-2.0 (`LICENSE`, `NOTICE`): permissive, with a patent grant, the licence ROS 2 uses. PX4, MAVROS and ROS 2 are not part of this repository. The OpenStreetMap files in `data/osm/` fall under the ODbL; the `README` and `NOTICE` give the required credit.
- **Secrets scan: done**. Before publishing (29 September 2026), every file in every version of the history, including the insides of compressed and zip files and the text of the PDFs, was searched for map tokens, Kaggle, GitHub, cloud and AI keys, private keys, passwords and e-mail addresses. The published repository was searched again on 7 October 2026: every file on `main` and every commit.
- **No key, token or password was found**, now or in the past. `config/map_keys.local.yaml` was never committed.
- The only hits are harmless: code that reads the keys from the private file, fake test values, and public shop phone numbers and e-mails that are part of the OpenStreetMap data.
- **Commit authors**. Every commit in the published history carries GitHub's no-reply address, not a personal e-mail address (checked on 7 October 2026).
- **Personal details in files**. About 390 tracked files, mostly raw logs, contain the laptop's home folder path with the user name. That is harmless; removing it would mean rewriting the logs.
- **Size**. A copy without the history is about 350 MB, mostly raw trial logs. That is fine for GitHub, but cloning is slow. Option: move the raw logs to a GitHub Release download and keep the summaries in the repository.
- **Responsible-use note: done** (a section in the `README`): civil research and education, simulation-tested only, real flights only with the flight test plan and legal permission, not for harm or weapons.
- **Automatic check and badge: done** (7 October 2026, section 1).
- **Still to do**: a `CONTRIBUTING.md` and issue templates, so that outside reports and changes arrive in a usable form.

3. Swarm logic: recommended improvements

1. **Stop leader flapping**. Today, a drone whose own link keeps dropping in and out takes the lead back every time it returns, because the lowest alive ID wins. This was seen in the first Phase 6 attempt with an overloaded radio (`docs/KNOWN_ISSUES.md`). - **Recommendation**: a hold-down rule. A drone that was counted as dead must be heard steadily, for example for 3 s (15 heartbeats), before it may claim the lead again. Until then it flies as a follower. - The lowest-ID rule and the term counter stay as they are. - **Cost**: this changes `docs/SPECIFICATION.md`, section 6, so the tests must be repeated. That means the 1,000 random runs of Phase 2 (minutes) and the PX4 trials of Phases 4 and 5 (about 8 hours with the queue). About 1 day of work plus the re-runs.
2. **Keep drones apart when they cannot hear each other**. In 143 of 1,000 random fast-simulator runs, two drones came closer than 5 m. Every one of those runs had a radio split or random link drops. - **Cheap first fix**: height layers during link loss. Each drone moves to its own height, for example 2 m apart by ID, while it has not heard its neighbours, so drones that cannot talk still cannot meet. - **Also**: "sticky" slots. A follower that loses the leader keeps its last slot and heading instead of drifting towards others. - **For real drones**: an onboard proximity sensor as the last layer. - About 1–2 days, then repeat the random runs and Phase 4 F2 and F5.

3. **Separation with the new RL policy.** On the Islamabad route, two drones once came to 4.77 m (limit 5 m). - Extend the brake so it also stops a drone that is closing in on a neighbour, not only on an obstacle. Or add a separation penalty to training (section 4). - Then re-run `scripts/rl_eval_route.py`. About half a day.
4. **Safer slot changes for the other shapes.** The line, column and echelon come closer than 5 m after faults, so they are experimental. Moving to a new slot on the transit layer, as the V does, should fix it. Re-run `scripts/formation_compare.py`. About 1 day.

4. Reinforcement learning: the next run

The two Kaggle runs (10.5 hours each) stopped improving after about 2.5 hours (`docs/RESULTS.md`). More hours with the same settings will not help. Change the set-up instead: - **Learning-rate decay:** lower the learning rate step by step to zero over the run, so the policy settles instead of going up and down. - **More dense courses:** dense clutter is still the weak case, with 18 of 30 missions passed. Use a curriculum from sparse to dense, or simply a larger share of dense courses. - **The brake inside training,** so the policy learns to work with it, and a **separation penalty** for coming closer than 5 m to a neighbour. - **Real map areas:** train on patches of real OpenStreetMap neighbourhoods as well as random courses. - **Three seeds** for a result others can trust. Because of the plateau, 5 hours per seed is enough: three seeds cost about 15 GPU hours, which fits in Kaggle's 30 free hours per week. - **One lesson from this run:** the uploaded zip must contain `reports/logs/rl/apf_tuning.jsonl`, or Kaggle's classical comparison runs untuned. Always re-test on the laptop (`docs/KAGGLE_GUIDE.md`, section 9). - **Choose the new policy by the same rules** (`docs/KAGGLE_GUIDE.md`, section 8), plus one more: closest pair at least 5 m on the real route.

Where RL should **not** be used: choosing the leader (it must be predictable and provable) and motor or attitude control (PX4 does this well).

5. Road to real drones

1. **Hardware per drone:** - a Pixhawk 6C with GPS; - a companion computer (for example a Raspberry Pi 5) running Ubuntu 24.04, ROS 2 Jazzy, MAVROS and this agent, wired to the Pixhawk by serial cable; - a telemetry radio; - a radio or Wi-Fi mesh for the heartbeats between drones; - an RC receiver with a kill switch.
2. **Onboard code in C++.** `cpp/avoid_policy/` holds the avoidance policy's inference in C++17 (observation, network, velocity correction), checked against the Python code on 100 reference inputs. Still to do: - wrap it in a ROS 2 `rc1cpp` node with the same topics as the Python agent, and fly both side by side in PX4 SIH until their velocity setpoints agree; - port the stopping-distance brake, which every result in `docs/RESULTS.md` uses; - time one step on the companion computer itself (`bench_avoid_policy`). About 2–3 days plus the comparison runs.
3. **A stable PX4 release.** PX4 is pinned to a release candidate, `v1.18.0-rc1`. Move to the stable release and repeat Phases 0–4 in simulation.
4. **A digital twin.** Measure the real drone (weight, battery, motors, a PX4 flight log) and put the numbers into the simulators, then repeat the tests.
5. **Bench test with the propellers off,** then one real drone with nine simulated ones. The set-up is `config/profiles/mixed.yaml`: only the connection address changes. Every step follows `docs/FLIGHT_TEST_PLAN.md`: geofence, RC override, kill switch, go/no-go checklist.
6. **Three real drones** in an open field with safety pilots; only then more.
7. **Legal permission first.** Check the current drone rules of the civil aviation authority (in Pakistan, the PCAA) and any local permission needed before buying or flying.

6. Keeping the project healthy

- **Versions:**
- ROS 2 Jazzy and Ubuntu 24.04 are long-term releases, supported until about 2029.
- Pin the Python packages in `requirements.txt` (section 1) so results can be repeated.
- Stay on PX4 `v1.18.0-rc1` until the stable release has been tested.
- **Evidence habit:** keep the rule that every claim comes with a log. The GitHub check repeats the fresh-copy test on every push; look at its result before every release.
- **Backups:** GitHub holds the code, history and tags. The Kaggle notebooks (`rafique/swarm-rl-train-seed1`, `-seed2`) hold their training output. The chosen policy and both result sets are already in the repository.

7. What to do first

#	Item	Why	Effort (estimate)	Who
1	Licence, secrets scan, responsible-use note, public release	nothing can be shared safely without these	done: public since 7 October 2026 (<code>v1.0.0</code>)	-
2	Level 1 one-click setup + automatic check	anyone can try the project in minutes	done: the check runs on every push since 7 October 2026	-
3	Hold-down rule against leader flapping	the one known weak point of the election	about 1 day + re-runs (about 8 h of PX4 trials)	owner approves the spec change
4	Height layers when drones cannot hear each other	the main separation risk before real flights	1–2 days + re-runs	development
5	Container and Codespaces (level 3)	the full PX4 system in one step on any computer	2–3 days	development
6	Next RL run with a changed set-up, 3 seeds	better in dense clutter; results others can trust	1 day of set-up + about 15 GPU hours	owner approves the quota
7	ROS 2 C++ node around <code>cpp/avoid_policy</code> , with the brake	onboard code without Python in the control loop	2–3 days + comparison runs in PX4 SIH	development
8	Real drones: hardware, stable PX4, bench test, flight plan	the goal of the project	weeks; needs hardware and permission	owner